

Advanced Data Mining: Homework set 7

2025/2026

Problem 1 (5 Points)

In this assignment, you will build and evaluate LSTM and GRU models for a one-step-ahead forecasting task on a financial time series. Use Python with TensorFlow (Keras) or PyTorch to implement the models. The tasks are:

- A. Data Preparation (0.5p): Load the weather time series dataset. For instance the Jena climate dataset, you can obtain it by running `curl https://www.bgc-jena.mpg.de/wetter/mpi_saale.2021b.zip -o mpi_saale.2021b.zip`. This is likely a multivariate time series (e.g., temperature, pressure, humidity, etc.). For the forecasting task, choose one target variable to predict (e.g., daily temperature). Define the forecasting horizon (e.g., predict the next 7 days of the target given the past 30 days of data). Split the data into training/validation/test segments by date.
- B. Implement LSTM Model (0.5p): Using your chosen framework, define an LSTM-based neural network for forecasting the next time step. (Tip: In Keras you can use `tf.keras.layers.LSTM`, and in PyTorch use `torch.nn.LSTM` within a custom `nn.Module`). Include necessary layers (e.g., one or two LSTM layers followed by a dense output layer). Choose a loss function such as Mean Squared Error and an optimizer (e.g., Adam). Train the LSTM model on the training set. Monitor the training and validation loss over epochs to ensure the model is learning (you may use early stopping to prevent overfitting).
- C. Implement GRU Model (0.5p): Define a similar neural network using Gated Recurrent Unit layers (`tf.keras.layers.GRU` or `torch.nn.GRU`). Ensure it has a comparable number of parameters to the LSTM model for a fair comparison (adjust the number of units if needed). Train the GRU model on the same training data, using the same training procedure and epochs as the LSTM.
- D. Evaluation (1p): For both models, forecast the next time step for each sequence in the test set. Compute error metrics on the test data, including Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).
- E. Hyperparameter Experiments (1p): Perform at least one hyperparameter variation for each model. For example, train a second LSTM with a different number of units or an extra LSTM layer, and note the impact on performance. Do the same for GRU (or vary learning rate, batch size, etc.). This will illustrate sensitivity to hyperparameters.
- F. Visualization (0.5p): Plot the model predictions vs. actual values on the test set for a visual comparison. Create a line plot for a subset of the test period (e.g., last 30 days) showing true vs predicted values for each model. This helps interpret where the model is performing well or failing (e.g., does it capture trends or miss sudden changes?).
- G. Interpretability Discussion (1p): Although LSTMs and GRUs are black-box models, examine their internal behavior if possible. For instance, you can plot the values of the LSTM forget gate over time for a test sequence to see how it modulates information, or analyze which input timesteps had the most influence on the prediction (e.g., by partial input perturbation). Discuss any insight gained about how the model is making decisions.

Baseline Benchmark: Compute a simple baseline forecast (e.g., “naive” forecast where the prediction is just the last observed value, or a basic linear regression on the last few lags). Compare the deep models’ performance to this baseline to quantify the value they add.